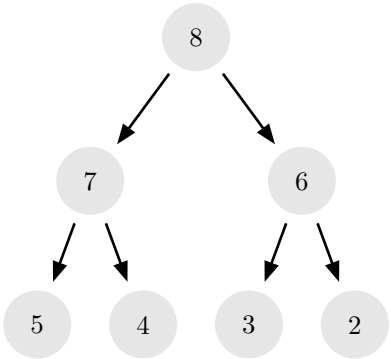
ARBOLES

1. Introducción

Algunas veces, cuando se modela una situación, se determina que la mejor manera de representar la relación que existe entre los objetos involucrados es mediante una estructura jerárquica. En estos casos se necesita una estructura adecuada que permita representar este tipo de estructuras jerárquicas, los árboles.
Un árbol es un grafo, conexo y acíclico.



Algunos autores tambien definen formalmente a un arbol como un conjunto T de nodos tal que:

1. Hay un nodo especialmente designado llamado la raiz del arbol, y
2. Los nodos restantes (excluyendo la raiz), son particionados en $m \geq 0$ conjuntos $T_1, ..., T_m$, donde cada uno de estos conjuntos es, a su vez, un árbol. Los árboles $T_1, ..., T_m$ se denominan subárboles de la raíz.

Esta ultima definición dada es recursiva, ya que se ha definido a un árbol en terminos de árboles, si bien hay formas no recursivas de definir un árboles, la definición recursiva parece la mas apropiada ya que la recursión es una caracteristica innata de lo árboles.

Representamos cada nodo del árbol como un objeto, de igual forma que las listas enlazadas, definimos que cada nodo contiene un atributo *info*, el resto de atributos van a ser los punteros a los otros nodos y varian en función del tipo de árbol.

1.1. Árboles N-Arios.

Formalmente, un árbol n-ario no ordenado con tipo base T se puede definir de manera recursiva como sigue:

1. La secuencia vacía representa al árbol nulo no ordenado. Se denota por Λ.
2. Sea e un elemento de tipo T y $A_1, A_2, ..., A_n$ árboles n-arios no ordenados con tipo base T. Entonces la secuencia $A =< e, [A_1, A_2, ..., A_n] >$, donde $[A_1, A_2, ..., A_n]$ denota un multiconjunto de árboles n-arios, es un árbol n--ario no ordenado on tipo base T y se definen las siguientes relaciones:
   - e es la raíz de A.
   - $\forall i (1 \leq i \leq n)$: A_i es un subárbol de A.
   - $\forall i (1 \leq i \leq n)$: e es padre de la raíz de A_i.
   - $\forall i (1 \leq i \leq n)$: La raíz de A_i es hijo de e.
3. Sólo son árboles n-arios no ordenados con tipo base T los objetos generados por la aplicación de las cláusulas (1) y (2).

En otras palabras, es un arbol donde un nodo puede tener muchos hijos.

1.1.1. *Implementación.*

Para la Implementación de los árboles n-arios, utilizamos la **representación hijo-izquierdo, hermano-derecho**, donde el árbol está representado por un apuntador a su nodo raíz y cada nodo tiene un apuntador a su primer hijo (hijo más a a izquierda) y a su hermano derecho.

Declaración de la clase NodoN

```
Class NodoN<Element>
  Attributes:
    Private:
      Element: info
      Pointer to NodoN<Element>: hijoIzq, herDer
  Methods:
    Public:
      proc construir(Element: info)
      proc construir(Element: info, Pointer to NodoN<Element>: hi, hd)
      func getInfo(): Element
      proc setInfo(Element: e)
      func getHijoIzq(): Pointer to NodoN<Element>
      func getHerDer(): Pointer to NodoN<Element>
      proc setHijoIzq(Pointer to NodoN<Element>: hi)
      proc setHerDer(Pointer to NodoN<Element>: hd)
FClass
```

Declaración de la clase ArbolN

```
Class ArbolN<Element>
  Attributes:
    Private:
      Pointer to NodoN<Element>: pRaiz
      int: peso
  Methods:
    Public:
      proc construir()
      proc construir(ref ArbolN<Element>: a)
      func getRaiz(): Element // O(1)
      func esNulo(): bool // O(1)
      func esHoja(): bool // O(1)
      func getPeso(): int // O(1)
      func getHijos(): Lista<ArbolN<Element>> // O(n)
      proc insertarSubArbol(ArbolN<Element>: subarbol) // O(n)
      proc eliminarSubArbol(int: pos) // O(n)
      func preorden(): Lista<Element> // O(n)
      func postorden(): Lista<Element> // O(n)
      func inorden(): Lista<Element> // O(n)
      func porNiveles(): Lista<Element> // O(n)
      func getHojas(): Lista<Element> // O(n)
      proc vaciar() // O(n)
      proc destruir() // O(n)
FClass
```

1.2. **Arboles Binarios.**

Un arbol binario, es otro tipo importante de la estructura de arbol, en el cual, cada nodo tiene a lo sumo dos subarboles, los cuales distinguimos como hijo izquierdo, e hijo derecho.

1.2.1. *Implementación.*

De forma similar a la representación de los arboles n-arios, un arbol binario va a tener un puntero a su nodo raiz, el cual tambien tiene su atributo *info*, se diferencia en el hecho de tener dos punteros, teniendo esta vez, uno a su hijo izquierdo y otro a su hijo derecho.

Declaración de la clase NodoBin

```
Class NodoBin<Element>
  Attributes:
    Private:
      Element: info
      Pointer to NodoBin<Element>: hijoIzq, hijoDer
  Methods:
    Public:
      proc construir(Element: info)
      proc construir(Element: info, Pointer to NodoBin<Element>: hi, hd)
      func getInfo(): Element
      proc setInfo(Element: e)
      func getHijoIzq(): Pointer to NodoBin<Element>
      func getHijoDer(): Pointer to NodoBin<Element>
      proc setHijoIzq(Pointer to NodoBin<Element>: hi)
      proc setHijoDer(Pointer to NodoBin<Element>: hd)
FClass
```

Declaración de la clase ArbolBin

```
Class ArbolBin<Element>
  Attributes:
    Private:
      Pointer to NodoBin<Element>: pRaiz
      int: peso
  Methods:
    Public:
      proc construir()
      proc construir(ref ArbolBin<Element>: a)
      func getRaiz(): Element // O(1)
      func esNulo(): bool // O(1)
      func esHoja(): bool // O(1)
      func getPeso(): int // O(1)
      func getHijoIzq(): Lista<ArbolN<Element>>
      func getHijoDer(): Lista<ArbolN<Element>>
      proc insertarNodo(Element: padre, e)
      proc eliminarNodo(Element: padre, e)
      func preorden(): Lista<Element> // O(n)
      func postorden(): Lista<Element> // O(n)
      func inorden(): Lista<Element> // O(n)
      func porNiveles(): Lista<Element> // O(n)
      func getHojas(): Lista<Element> // O(n)
      proc vaciar() // O(n)
      proc destruir() // O(n)
FClass
```

2. Recorridos

Una idea que aparece repetidamente en los algoritmos de arboles, es la noción de atravesar o recorrer un árbol. Este es un metodo de examinar los nodos de el árbol sistematicamente para que cada nodo sea visitado exactamente una vez. Un recorrido completo de un arbol nos da una redisposicion lineal de los nodos.

Hay cuatro formas principales para recorrer un arbol, estas son, preorden, inorden, postorden y por niveles (tambien existe por ramas), los primeros tres recorridos son definidos recursivamente, el ultimo se diferencia en ser iterativo y necesitar una estructura auxiliar (generalmente una cola).

2.1. Preorden.

2.1.1. *Pasos.*

- Procesar la raiz
- Recorrer los hijos en preorden

2.1.2. *Implementación.*

Metodo publico del recorrido preorden, se encarga de construir la lista resultado y iniciar el recorrido

```
Func ArbolN<Element>::preorden(): Lista<Element>
    Var
        Lista<Element>: result
        Pointer to NodoN<Element>: ptr
    Begin
        result.construir()
        ptr ← instance.pRaiz
        instance.preorden(ptr, result)
        Return result
FFunc
```

Metodo protegido del recorrido preorden, se encarga de recorrer el arbol y añadir los elementos a la lista resultado

```
Proc ArbolN<Element>::preorden(Pointer to NodoN<Element>: ptr,ref Lista<Element>: result): Lista<Element>
Begin
    if ptr ≠ null then
        result.insertar(ptr↑.getInfo(), result.getLongitud() + 1)
        ptr ← ptr↑.getHijoIzq()
        while ptr ≠ null then
            instance.preorden(ptr, result)
            ptr ← ptr↑.getHerDer()
        EndWhile
    endif
FProc
```

2.2. Inorden.

2.2.1. *Pasos.*

- Recorrer el primer hijo en inorden
- Procesar la raiz
- Recorrer el resto de los hijos en inorden

2.2.2. *Implementación.*

Metodo publico del recorrido inorden, se encarga de construir la lista resultado y iniciar el recorrido

```
Func ArbolN<Element>::inorden(): Lista<Element>
    Var
        Lista<Element>: result
        Pointer to NodoN<Element>: ptr
    Begin
        result.construir()
        ptr ← instance.pRaiz
        instance.inorden(ptr, result)
        Return result
FFunc
```

Metodo protegido del recorrido inorden, se encarga de recorrer el arbol y añadir los elementos a la lista resultado

```
Proc ArbolN<Element>::inorden(Pointer to NodoN<Element>: ptr,ref
Lista<Element>: result): Lista<Element>
Begin
    if ptr ≠ null then
        instance.inorden(ptr↑.getHijoIzq(), result)
        result.insertar(ptr↑.getInfo(), result.getLongitud() + 1)
        ptr ← ptr↑.getHijoIzq()↑.getHerDer()
        while ptr ≠ null then
            instance.inorden(ptr, result)
            ptr ← ptr↑.getHerDer()
        EndWhile
    endif
FProc
```

2.3. Postorden.

2.3.1. *Pasos.*

- Recorrer los hijos en postorden
- Procesar la raiz

2.3.2. *Implementación.*

Metodo publico del recorrido postorden, se encarga de construir la lista resultado y iniciar el recorrido

```
Func ArbolN<Element>::postorden(): Lista<Element>
  Var
    Lista<Element>: result
    Pointer to NodoN<Element>: ptr
  Begin
    result.construir()
    ptr ← instance.pRaiz
    instance.postorden(ptr, result)
    Return result
FFunc
```

Metodo protegido del recorrido postorden, se encarga de recorrer el arbol y añadir los elementos a la lista resultado

```
Proc ArbolN<Element>::postorden(Pointer to NodoN<Element>:
ptr,ref Lista<Element>: result): Lista<Element>
Var
  Pointer to NodoN<Element>: aux
Begin
  if ptr ≠ null then
    aux ← ptr↑.getHijoIzq()
    while aux ≠ null then
      instance.postorden(aux, result)
      aux ← aux↑.getHerDer()
    EndWhile
    result.insertar(ptr↑.getInfo(), result.getLongitud() + 1)
  endif
FProc
```

2.4. **Por niveles.**

2.4.1. *Implementación.*

Implementación del recorrido por niveles

```
Func ArbolN<Element>::porNiveles(): Lista<Element>
Var
    Cola<Pointer to NodoN<Element>>: colaAux
    Lista<Element>: result
Begin
    result.construir()
    colaAux.construir()
    colaAux.encolar(instance.raiz)
    while ¬colaAux.esVacia() do
        hijo ← colaAux.getFrente()↑.getHijoIzq()
        while hijo do
            colaAux.encolar(hijo)
            hijo ← hijo↑.getHerDer()
        EndWhile
        result.insertar(colaAux.getFrente()↑.getInfo(), result.getLongitud() + 1)
        colaAux.desencolar()
    EndWhile
    Return result
FFunc
```